

Designing like a UI Engineer

Lesson 30 min

Somewhat ironically, or at least counterintuitively—translating a visual UI design from mockups in a tool like Figma into a buildable software interface, is one of the most complex tasks in developing a piece of software, even though what it yields for users is the most concrete.

I say this is counterintuitive, because for most designers, looking at a UI in a visual design tool feels very concrete and intuitive, and the task of “making it work” feels like it should be a process of “making the interface look like that”. After all, if we’ve already done the work of making the UI look a certain way using one tool—especially if we’ve laboriously taken the time in that tool to structure our designs into components, styles, and variables—how hard can it be to translate from that tool to code?

I say this is ironic, because among developers, implementing a UI is often tossed off as a lesser skill than “hardcore” ones like algorithms, performance, and dev ops; and because software development firms have systematically devalued the role of UI implementation over the past decade by folding this work in with generalist “full-stack” developer jobs, and reducing pay scales for specialized roles in this area so that they may fall to someone more junior.

The result is that there are a lot of poorly-built user interfaces out there, far more bad than good, resulting in a lot of buggy user interfaces and sloppy UI code. Large Language Models, in turn, have been trained on large volumes of codebases with poor approaches to user interface code. There are efforts to train these tools to write better coding by giving them access to take screenshots of rendered code so that an image-recognition model can compare the results to a reference—and this probably helps, but visual fidelity is not the only issue with poor UI code, and not the only challenge in building user interfaces.

The main reason that user interfaces represented in a visual design tool don’t translate easily to code is more like a system mismatch. Visual design tools allow us to visually describe the overall expected output of a program which renders a graphical user interface, but expressing interfaces in code requires an underlying systematic approach that is difficult to infer from visual output only.

In other words, building a user interface from a visual reference requires an act of interpretation, not just reproduction, a form of interpretation which most developers

struggle with at best, and which LLMs can only extrapolate about.

So for now, as we start building apps, I have to ask you to quiet a bit some of your usual designer instincts, especially the ones geared toward visual design. Instead, I'm going to ask you to try to reason about interface design from a different part of your brain, we'll call it the engineer or developer part, to build up your mental muscles for structures and systems.

Instead of starting from a visual design reference and working backward to the code, let's start from the code and build up our way to an emergent interface design.

This is roughly the process I recommend for starting to work this way:

- ① Start with a skeleton of basic (built-in) UI elements to make something functional but only.
- ② Shape your elements into an overall layout structure.
- ③ Work your way in, laying out elements within layout regions, until you've got a rough layout.
- ④ Encode an initial palette of values for fonts, colors, sizing, and spacing.
- ⑤ Apply basic visual styles to elements, using as much shared code as possible—make all repeated elements look the same, and tie all of your styles to your palettes.
- ⑥ Add in visual differentiation (variants) for elements that should contrast with each other for some reason, but otherwise behave the same. Extend your palettes as needed during this phase.
- ⑦ Work on visual details to polish and finish your UI.
- ⑧ If you get stuck at any point, accept the possibility of limitations (even unknown ones) and consider design choices that may be easier to implement.

As with any process, this approach is inevitably non-linear: polishing details may lead to rethinking the layout, for example. But the goal with this layered approach remains: to produce UI code that is more robust and easier to reason about and make fine-tuning adjustments to at the end.

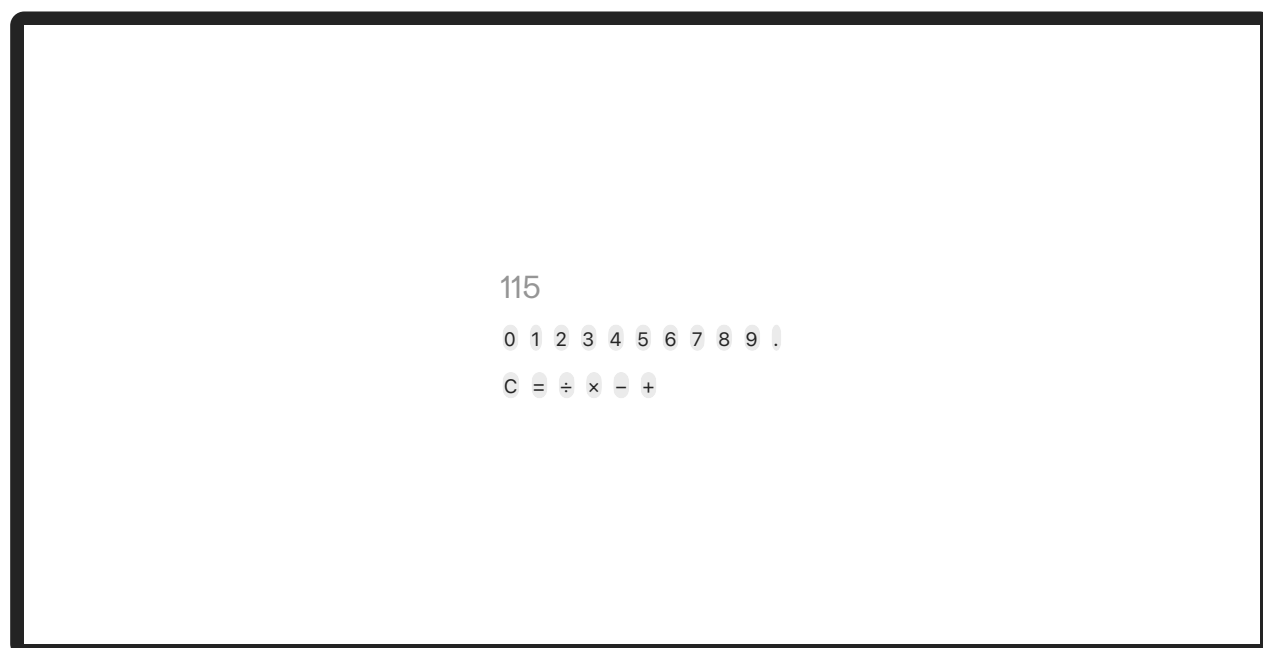
Building the calculator example

Let's look at an example of this by building a UI for a four-function calculator. We'll walk through the same process step by step so you can see the engineer's approach in action.

First: a basic skeleton

Step 1 of our process is to start with a skeleton of basic, built-in UI elements—

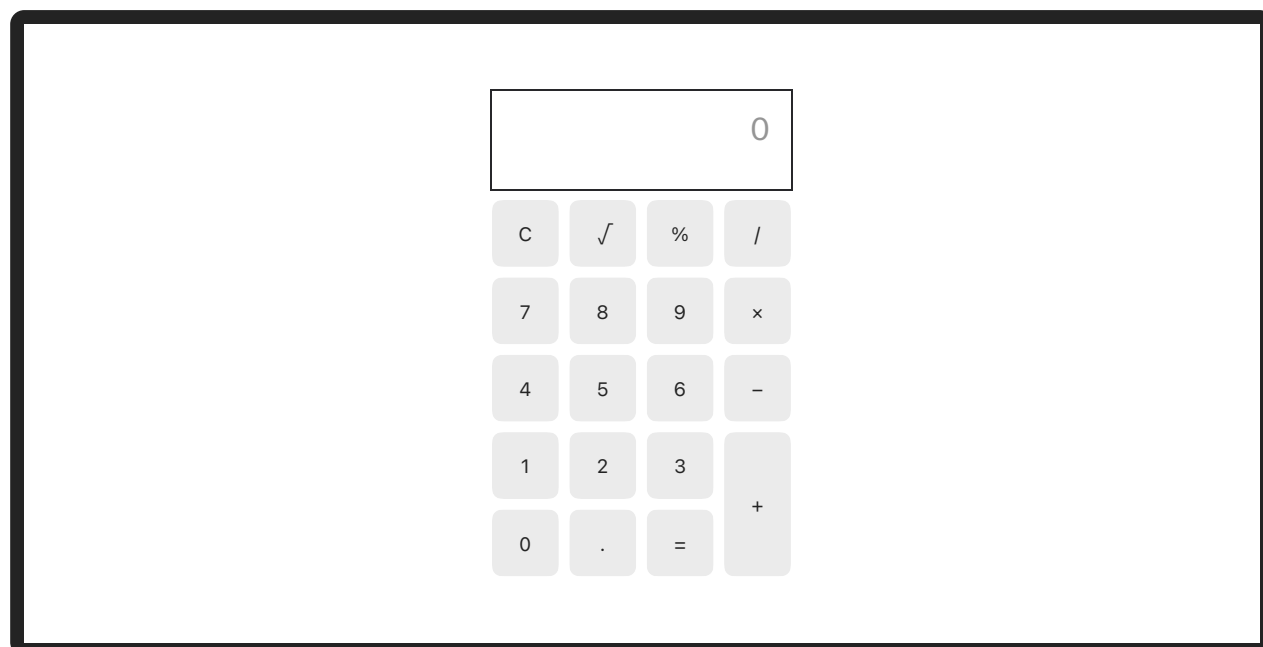
something functional only, with no attempt at visual polish yet. Here's a calculator that does the math using nothing fancier than buttons and a display.



Function without much form: here we are leaning heavily on the built-in UI elements and the browser's default styles. This gives us a better starting point to drive the design.

Shaping the layout

Next we shape those elements into an overall layout structure and work inward into the regions (steps 2 and 3). We're not styling yet—just establishing where things sit.



As we laid out the calculator buttons, we also made some adjustments to the available calculator functions. That way we could lay them out in a familiar grid pattern.

A design palette

Before applying visual styles, we encode an initial palette of values for fonts, colors, sizing, and spacing (step 4). This gives us more control over the look and feel later,

and keeps our styles tied to **tokens** instead of magic numbers.

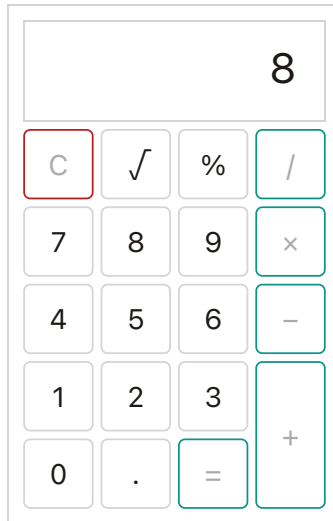
Colors	Font	Sizing	Spacing	
--color-neutral-100	The quick brown fox	--size-1	--space-1	
	--font-sans	--size-2	--space-2	
--color-neutral-300	The quick brown fox	--size-3	--space-3	
	--font-serif		--space-4	
--color-neutral-500				
--color-neutral-700				
--color-neutral-900				
--color-teal-500				
--color-red-600				

The palette will be in code, but here for convenience we’re visually presenting it.

At this stage, it’s perfectly okay to start with a basic palette. The point is to build some control structures into the foundation of the design, for better adjustment later on. It’s almost impossible to imagine how a design palette will work until you start applying it to the design.

Applying the palette to the calculator

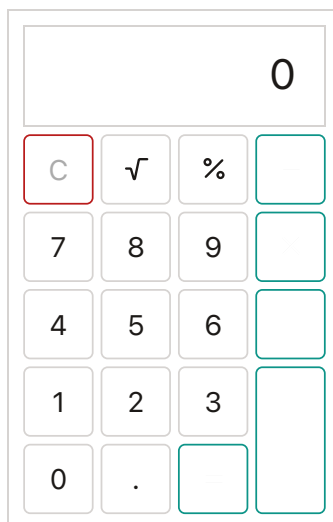
Step 5: we apply basic visual styles to the calculator, using as much shared code as possible and tying everything to our **design palette**. Repeated elements look the same; consistency comes from the system.



Now we can see the design palette in action.

Icons as polish

For step 7—visual details and polish—we swap the operator characters for inline SVG icons. That way color and style still come from CSS instead of being baked into the glyphs, and we keep one source of truth for appearance.



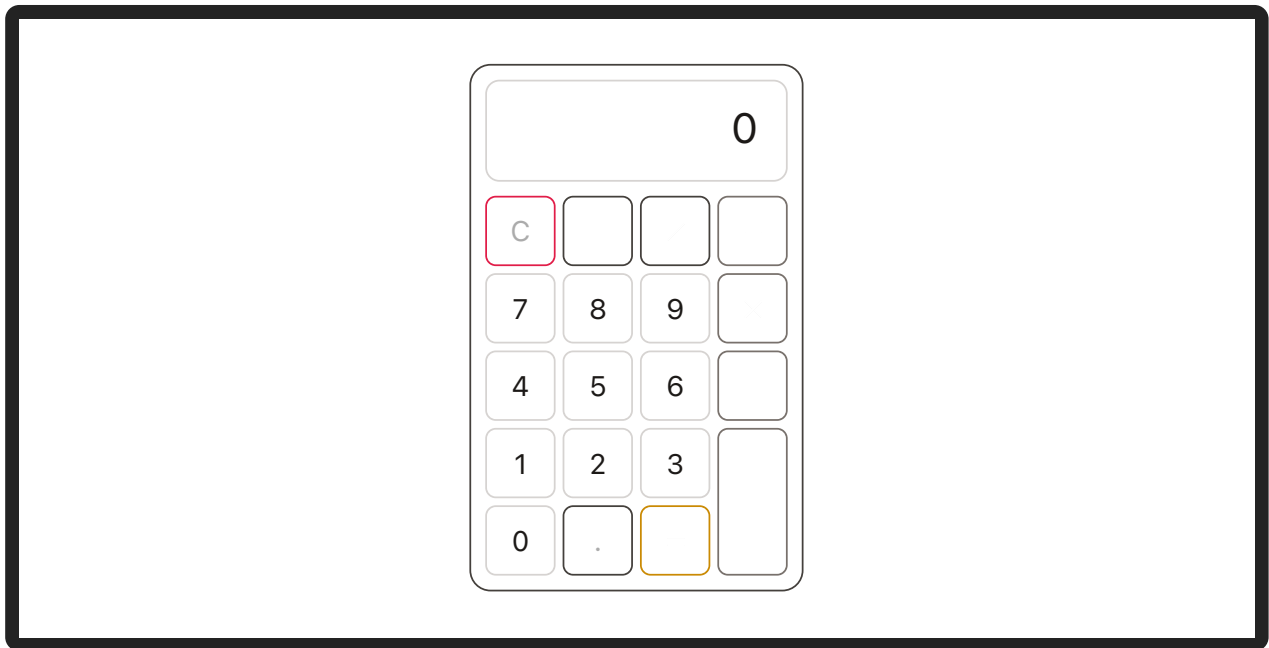
These icons should give a better shape to the operators compared to the text characters we started with.

It's a bit out of scope for this lesson, but in general it's best to bring in an icon library if you need icons. In this case, we used the open source Lucide icon set.

Visual differentiation

Step 6 is about adding **visual differentiation** (variants) for elements that should

contrast—here, different groups of controls (numbers, operators, functions). We extend our palettes as needed so the system stays coherent and manageable.



At this stage, the benefits of the design palette and giving elements semantic names become more apparent, because design ideas like “I think the calculator body should use the darkest gray” or “The number buttons should be the lightest, followed by the main operators, etc.” now map more clearly to the code.

Refining the palette

Along the way to the latest design above, we also refined the **design palette** (tokens) to support the differentiated groups. That gives us something easier to work with and tweak later.

Base Colors	Semantic (usage)	Font
☐ --color-white	☐ --calc-body-bg	The quick brown fox
☐ --color-neutral-100	☐ --calc-body-border	--font-sans
☐ --color-neutral-200	☐ --calc-body-text	
☐ --color-neutral-300	☐ --calc-display-bg	Sizing
☐ --color-neutral-400	☐ --calc-display-border	--size-2
☐ --color-neutral-500	☐ --calc-display-text	--size-3
☐ --color-neutral-600	☐ --calc-digit-bg	
☐ --color-neutral-700	☐ --calc-digit-text	Spacing
☐ --color-neutral-800	☐ --calc-digit-border	--space-1
☐ --color-neutral-900	☐ --calc-operator-bg	--space-2
☐ --color-raspberry	☐ --calc-operator-fg	--space-3
☐ --color-gold	☐ --calc-utility-bg	Radius
	☐ --calc-utility-fg	--radius-sm
	☐ --calc-clear-bg	--radius-md
	☐ --calc-equals-bg	--radius-lg
	☐ --calc-action-fg	

That's the process in practice: we started from structure and system, not from a static mockup. Each step builds on the last, and the result is UI code that's easier to reason about and adjust. When you build your own interfaces, you can use this same sequence—and when you get stuck, remember step 8: consider design choices that might be easier to implement.

Design tokens in practice

In the calculator example above, you saw how a **design palette** (encoded as CSS custom properties) gives you a layer of control between your design intentions and

the raw code. This idea—named values that represent design decisions—is what the industry calls **design tokens**.

When you work with LLM coding tools, tokens become even more powerful. Instead of telling the LLM “change the hex value on line 47,” you can say things like:

“Make the base palette warmer”

“Increase the spacing scale by about 20%”

“Switch the accent color to something in the blue-green range”

These are the kinds of design directions you’d give to a colleague. When the code is organized around named tokens, the LLM can find the right things to change—and the result is more predictable than asking it to hunt through scattered hardcoded values.

You’ll practice extracting and using design tokens hands-on in Week 2, in the Refactoring for Design Control lesson.

Naming things: shared vocabulary with the LLM

One more thing worth mentioning. When you’re working with an LLM on design changes, it helps to establish a **shared vocabulary** for the different parts of your interface. If you call something a “header bar” and the LLM calls it a “top navigation container,” you’ll talk past each other.

Try asking: “What would you call the different UI elements in this project?” The LLM will come back with names—maybe something like “control bar,” “score display,” “button group.” If those names make sense to you, start using them in your prompts. If they don’t, suggest better ones. The point is to get on the same page so your design directions are more precise.

This is a design skill, not just a coding skill. Naming things clearly—giving every element a role and identity—is how you build an interface that’s coherent and maintainable, whether you’re working in Figma, in code, or with an LLM collaborator.

Design tokens aren’t just visual. Interface strings—button labels, headings, error messages—could also be separated into a central reference. If you’ve ever worked with a content design system or internationalization (i18n), it’s the same idea: one source of truth for the words your interface uses.

Applying this to your Score Keeper

When working on your Score Keeper project, try applying this process to the UI. You don't need to follow every step exactly—the goal is to practice thinking in layers:

- ① Get the structure working first (basic HTML, default browser styling)
- ② Shape the layout before worrying about colors
- ③ Define a small set of tokens for your design values
- ④ Apply styles that reference those tokens
- ⑤ Differentiate elements that need to stand out
- ⑥ Polish at the end

If your Score Keeper is already built and styled, that's okay too—you can always refactor what you have. In Week 2, we'll walk through a refactoring process step by step before building the Pomodoro Timer.

Feedback

I'd love to hear your feedback about the course. It's not required, but highly appreciated.

How did you like this part? What could be better?

Send feedback

Mark incomplete

UP NEXT

🕒 Score Keeper

Project 3 hours

